

Decision standard: every default has an owner, an enforcement point, a verification command, and a recovery route - no tribal knowledge required.

1. SURFACE STRATEGY

- **DECISION 1** - Billing defaults to the Codex app for task orchestration and reviewable handoffs. IDE extension is for small local edits; CLI is for repeatable/scripted or CI-adjacent work. Cloud is a governed exception for eligible long-running, non-sensitive tasks.
- **DECISION 2** - Worktrees are mandatory for payments, migrations, shared libraries, release branches, and parallel tasks; optional only for isolated docs, tests, and single-file experiments.
- **DECISION 3** - Cloud is required for approved long-running parallel work that benefits from managed isolation; forbidden for production access, customer/regulated data, unapproved secrets, incident response, or repos without cloud eligibility.
- **GUARDRAIL 1** - Windows sandbox defaults to unelevated. Elevated mode is only for vetted setup needing installs/drivers plus an approved admin path. Use WSL semantics for Linux-first tooling. If elevated setup is blocked, use unelevated + prebuilt dev image/remote runner or WSL; do not bypass controls.

4. GOVERNANCE BUNDLE

- **Repo root:** version AGENTS.md with verification commands, review guidance, ownership boundaries, and a concrete “done” definition (tests green, diff reviewed, docs/changelog as required, no untracked secrets). Store organization defaults in managed config.toml; repo-specific overrides stay checked in and reviewed.
- **GUARDRAIL 2** - .rules allowlists safe build/test/git-read commands; approval is required for writes outside the workspace, package installs, network, credentials, privileged operations, destructive git, and production tooling.
- **Publish managed skills** for secure PR review, CI fix loop, release notes, dependency updates, and incident-safe triage. Version, test, and assign owners to each skill.

2. MODEL STRATEGY

- **DECISION 4** - Default to GPT-5.6 Terra for most engineering work. API-key workflows use the current API flagship GPT-5.6 (gpt-5.6-sol) for complex coding/reasoning; pin tested snapshots where repeatability matters. Revalidate availability at rollout time.
- **Escalate reasoning** from medium to high/xhigh for cross-repo changes, ambiguous failures, migrations, security-sensitive review, or when two bounded attempts fail. Keep routine edits and known test fixes at lower effort.
- **Spark:** use only for ultra-low-latency, narrow edit-test loops with a small diff and deterministic checks. Do not use it for design, multi-file refactors, security, migrations, or tasks needing broad context.
- **Spark guardrail:** tighten scope to one intent, require explicit target files, run the relevant test/lint command, and have a human inspect the diff before merge.

5. CLOUD + INTEGRATIONS

- **GUARDRAIL 3** - Cloud environments use a minimal pinned setup script and lockfile-hash caches. Secrets come only from a secrets manager with short-lived, task-scoped credentials - never repo files, prompts, or durable plain environment variables. **GUARDRAIL 4** - Internet is off by default; enable only destination allowlists per task. GitHub: start with manual @codex review for Billing Platform, required human approval, and sampling-based calibration. Enable automatic reviews only after quality, false-positive, and access thresholds are met.
- **Slack/Linear:** delegate status drafting, issue summaries, acceptance-criteria proposals, and PR/CI links. Never delegate approvals, incident command, production changes, customer commitments, security exceptions, or external posting without a named human owner.

3. CONTEXT CONTINUITY

- **DECISION 5** - Checkpoint at each completed subgoal, before risky commands, after a material decision, every 20-30 active minutes, and before switching surfaces. Record goal, decisions, changed files, verification, next command, and blockers in the task template.
- **Compact** only after writing the checkpoint. Preserve accepted/rejected alternatives, constraints, exact environment/sandbox state, and links to PR/issue/logs; remove only redundant tool chatter.
- **RECOVERY A** - failed attempt: capture error + revert only owned changes -> retry once with a narrower hypothesis -> escalate to human. **RECOVERY B** - drift: re-read instructions/diff and restore the stated goal. **RECOVERY C** - surface switch/restart: open the checkpoint, confirm git status and environment, then continue in a new thread if context is compromised.

6. ADMIN + MONITORING

- **RBAC:** separate local user, cloud-eligible developer, repo steward, auditor, and admin groups. Start Billing Platform with a small champion cohort; grant cloud only to approved groups and repositories.
- **Track analytics:** adoption, surface/model mix, task completion, latency, retry/failure rate, review acceptance, and support volume. Track compliance logs separately: identity, repo/environment, approvals, sandbox mode, network/secrets policy events, tool actions, and retention/export access.
- **Support runbook:** inspect approval history, sandbox mode, selected environment/image, policy match, identity/sign-in entitlement, repo/cloud eligibility, and missing GitHub/Slack/Linear permissions. Windows triage: verify elevated prerequisites, active unelevated/elevated sandbox mode, WSL availability if selected, and corporate sign-in/entitlement path.

ROLLOUT GATES

1 Billing pilot: local + worktrees + manual review

2 Expand after policy/test evidence

3 Cloud + automation only for eligible cohorts

Owner: Platform Engineering | Review cadence: weekly during pilot; monthly after scale-out | Decision log: attach to rollout PRD